

Study Memo: Concepts and Methodologies

Forecasting Daily Equity Volatility: Can Machine Learning Beat Classical Models?

(Trimester 9 Term Project)

François Schmitt (Roll Number: 23035010523)
f.schmitt@op.iitg.ac.in

Standard version

This memo explains every concept and methodological choice in the project, in the order the pipeline uses them, and ends with sample questions and answers of the kind an examiner might ask. A two-page condensed version exists as `memo_condensed.tex`. All numbers refer to the final run on data through 2026-06-30.

1 The problem setting

Volatility. The conditional standard deviation σ_t of an asset's return: how large tomorrow's price swing is expected to be. It prices options, sets value-at-risk limits and margin requirements, and determines position sizes, so forecasting it one day ahead has direct practical value.

Log returns. $r_t = \ln(P_t/P_{t-1})$, modelled as $r_t = \sigma_t z_t$ with z_t zero-mean, unit-variance noise. Log returns aggregate additively across days and are the object GARCH-type models are written for.

The task. One-day-ahead *variance* forecasting: at the close of day $t - 1$, output $\hat{\sigma}_t^2$ using information up to day $t - 1$ only.

Latent volatility and the proxy. True volatility is never observed on any day. Forecasts are scored against the squared return r_t^2 , which is conditionally unbiased ($\mathbb{E}[r_t^2 | \mathcal{F}_{t-1}] = \sigma_t^2$) but extremely noisy: a calm day can print a large return and a wild day a small one. Everything in the evaluation design flows from this noise.

The research question. Classical econometrics forecasts volatility with small recursive models (GARCH family). The project asks whether machine learning (gradient-boosted trees) and deep learning (LSTM) can beat well-implemented classical baselines under a fair, leak-free protocol, judged by a noise-robust loss and a significance test.

2 The stylized facts the models exploit

Unpredictable direction. The autocorrelation function (ACF) of raw returns is flat: past returns do not predict the sign of the next one. There is no free lunch in the mean.

Volatility clustering. The ACF of absolute returns $|r_t|$ decays slowly and stays significant for dozens of lags: calm days follow calm days, turbulent days follow turbulent days. This persistence in magnitude is the only signal any model in the study exploits.

Leverage effect. Negative returns raise future volatility more than positive returns of the same size (falling prices increase a firm's leverage and markets fear drawdowns). GJR-GARCH and EGARCH encode it explicitly; the learned models can discover it from signed return lags.

3 The evaluation harness

Walk-forward, no look-ahead. The forecast for day t may use information up to day $t - 1$ only, everywhere, with no exception. All models share one split: the final 30% of observations is the out-of-sample test

window. On the final data this is 4,650 returns (2008-01-03 to 2026-06-29), split into 3,255 training days (to 2020-12-04) and 1,395 test days (2020-12-07 to 2026-06-29). The COVID crash falls in training; the test window contains the 2022 bear market and the April-2025 tariff shock, two sharp asymmetric drawdowns.

Periodic refit, daily roll. Estimated models are re-fit every 22 trading days (about monthly) on an expanding window; between refits their recursions are rolled forward one day at a time using observed returns. This is both fast and valid: parameters never see the future, and daily updates use realized data only.

4 Classical baselines

Naive. $\hat{\sigma}_t^2 = r_{t-1}^2$: tomorrow’s variance is today’s squared return. Unbiased but maximally noisy.

Rolling window. The variance of the previous 21 returns, the “one-month historical vol” of every trading desk.

EWMA (RiskMetrics). $\hat{\sigma}_t^2 = \lambda \hat{\sigma}_{t-1}^2 + (1 - \lambda) r_{t-1}^2$ with $\lambda = 0.94$: an exponentially weighted average of past squared returns. One fixed parameter, no estimation, hard to beat.

GARCH(1,1). The workhorse recursion

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2, \quad (1)$$

fit by maximum likelihood (zero mean, normal innovations). α reacts to news, β carries persistence; $\alpha + \beta$ close to one reproduces volatility clustering with three parameters.

GJR-GARCH. Adds the threshold term $\gamma r_{t-1}^2 \mathbf{1}[r_{t-1} < 0]$: a negative return contributes extra variance. The leverage effect as an additive switch.

EGARCH. Models $\ln \sigma_t^2$ with a signed standardized-residual term, so leverage enters through the sign of z_{t-1} and positivity of the variance is guaranteed by construction, with no parameter constraints.

Implementation honesty. The daily recursions rolled between refits were verified line by line against the arch package’s own internal recursion source, so the hand-rolled updates are mathematically identical to what the fitted parameters imply. Returns are scaled by 100 before fitting for numerical stability of the likelihood and forecasts rescaled after.

5 The learned models

Shared features. At row t , using data through $t - 1$ only: lagged returns and squared returns (lags 1–10), realized variance over the past 5 and 21 days (the daily/weekly/monthly horizons of the HAR literature), and the 5-day mean absolute return as an outlier-robust volatility level. Signed return lags let the models learn the leverage effect on their own.

Shared target. Regressing on raw r_t^2 fails: the proxy is so noisy that a squared-error learner collapses toward a constant. Both learned models instead predict

$$y_t = \ln \left(\frac{1}{5} \sum_{j=0}^4 r_{t+j}^2 + \varepsilon \right), \quad (2)$$

the log of forward 5-day average squared returns: averaging suppresses proxy noise and the log makes the target approximately Gaussian. Because y_t is only fully observed at $t + 4$, every training set excludes a 5-day buffer of the most recent rows. This is the honesty cost of a forward-looking target.

XGBoost, walk-forward. A gradient-boosted ensemble of shallow trees (400 trees, depth 3, learning rate 0.03, row and column subsampling 0.8, a deliberately conservative configuration) re-trained from scratch every 22 days on all fully realized targets, mirroring the GARCH refit cadence so the comparison is fair. Warm-starting each refit from the previous booster was evaluated and rejected: it changes the ensemble’s composition for negligible savings, sacrificing run-to-run reproducibility.

LSTM. A single-layer recurrent network (32 hidden units, linear head) reads the previous 20 days of standardized features and predicts y_t . The anti-leakage guards are explicit: each sequence is labelled by

the target at its *last* row, so no sequence contains rows after its own label date; feature standardization uses training-window statistics only; the early-stopping validation slice is carved from the *end of the training window*, never from test. Training is Adam on MSE in log-variance space, mini-batches of 64, up to 200 epochs with patience 15 and best-checkpoint restoration, fixed seed. On the final data it early-stopped at epoch 37. The architecture is deliberately small: roughly 2,800 noisy training sequences cannot support a deep stack.

6 Evaluation criteria

RMSE and MAE. Computed on the volatility scale ($\sqrt{\text{variance}}$), against $|r_t|$, purely for interpretability: an RMSE of 0.007 means “typically about 0.7 volatility points off per day”. Symmetric losses: they price over- and under-prediction equally.

QLIKE. The headline criterion,

$$\text{QLIKE} = \frac{1}{T} \sum_t \left(\frac{p_t}{v_t} - \ln \frac{p_t}{v_t} - 1 \right), \quad (3)$$

with proxy $p_t = r_t^2$ and forecast $v_t = \hat{\sigma}_t^2$. Two properties make it the right loss. First, robustness: Patton (2011) showed that QLIKE (and squared error on variance) are the losses whose model *rankings* are unaffected by mean-zero noise in an unbiased proxy, so the noisy r_t^2 still ranks models correctly. Second, asymmetry: QLIKE punishes under-prediction of variance far more than over-prediction, which is exactly the asymmetry a risk manager wants, since being under-hedged in a crash costs more than being over-hedged in a calm.

Why the naive baseline’s QLIKE explodes. QLIKE contains p_t/v_t ; when a near-zero variance forecast (yesterday was flat) meets a non-zero realization, the ratio diverges. The naive forecast produces such near-zero forecasts routinely, hence its QLIKE of 5.4×10^3 against roughly 1.5 for everything else. A property of the loss, not a bug in the code.

Diebold–Mariano test. Given two models’ per-day loss series, DM tests whether the mean loss difference is statistically distinguishable from zero, with the Harvey–Leybourne–Newbold small-sample correction. Without it, a leaderboard gap of 0.01 QLIKE is just noise-reading. Negative statistic means the first model has lower loss.

7 Results and how to read them

- **Winner:** EGARCH(1,1), QLIKE 1.5042, statistically tied with GJR-GARCH at 1.5091 (DM $p = 0.468$). The two leverage parameterizations are interchangeable on this window.
- **Leverage is now significant:** EGARCH versus plain GARCH(1,1) gives DM statistic -4.211 , $p < 0.0001$. On the shorter sample ending 2024 the same comparison was insignificant ($p = 0.87$) and plain GARCH ranked first: the leverage effect earns its keep exactly when the test window contains asymmetric crashes (2022 bear market, April-2025 tariff shock).
- **The LSTM is the best learned model:** third overall at 1.5287, ahead of plain GARCH (1.5569), and statistically indistinguishable from the winner ($p = 0.193$). Its signed return lags let it learn the leverage pattern from data.
- **Point accuracy versus QLIKE:** the LSTM has the best RMSE (0.00686) and XGBoost the best MAE (0.00496), yet both trail on QLIKE. The smoothed target (2) makes them accurate on average but slow at spikes, and QLIKE charges heavily for arriving late to an eruption.
- **Verdict:** no learned model outranks the leverage GARCH family on the risk-relevant criterion, so the answer to the research question remains no, but a narrower no than on shorter samples.

8 Why the classical family stays ahead

Three structural reasons. First, maximum likelihood estimates the GARCH family against the correct variance objective, while the learned models train on the smoothed surrogate (2), which biases them toward smoothness. Second, roughly 3,200 noisy daily observations is a small-data regime that favours 3–4 parameter recursions over networks; published deep-learning wins typically involve intraday realized-variance data, many assets, or exogenous inputs. Third, QLIKE’s asymmetry rewards reacting to a single large return the very next day, which is precisely what a GARCH recursion does.

9 Limitations, honestly stated

Single asset (S&P 500) and single test window, so conclusions do not generalize to ML-for-volatility at large. The squared-return proxy is the noisiest admissible choice; intraday realized variance would sharpen both training and evaluation. Hyperparameter search was deliberately modest, and with the LSTM already within statistical noise of the winner, tuning could plausibly close the remaining gap. One seed per learned model. The DM test between the two best models selected on the same test set carries a mild selection bias; the EGARCH-versus-GARCH comparison is unaffected by it.

10 Sample questions and answers

Q. *Why forecast volatility rather than returns?*

A. Because volatility is forecastable and returns are not. The ACF of returns is flat (no linear predictability in the mean), while the ACF of $|r_t|$ decays slowly and stays significant for dozens of lags. Removing the sign reveals the persistence; that contrast is the empirical foundation of the whole project.

Q. *True volatility is never observed. How can you evaluate forecasts at all?*

A. Against a proxy. The squared return is conditionally unbiased for the variance, so with a loss whose ranking is robust to proxy noise (QLIKE), models can be ranked correctly even though each day’s proxy is very noisy. This is Patton’s (2011) result and the reason QLIKE is the headline metric.

Q. *Why is QLIKE preferred over RMSE here?*

A. Two reasons. RMSE on a noisy proxy can re-rank models spuriously, while QLIKE’s ranking is provably robust to unbiased proxy noise. And QLIKE is asymmetric: it penalizes under-predicting variance far more than over-predicting it, matching the risk manager’s real loss. The project’s results show why this matters: the learned models win RMSE and MAE yet lose QLIKE.

Q. *Why do the learned models not predict r_t^2 directly?*

A. Because a squared-error learner trained on so noisy a target collapses toward a constant. Both learned models predict the log of a 5-day forward average of squared returns instead: averaging cuts the noise, the log makes the target roughly Gaussian. The cost is honesty overhead: the target for day t is only known at $t + 4$, so training sets exclude a 5-day buffer of the freshest rows.

Q. *Why walk-forward evaluation instead of cross-validation?*

A. Time series break the exchangeability that cross-validation assumes: a random split trains on the future and tests on the past, leaking the regime. The walk-forward protocol reproduces deployment: forecasts for day t use data through $t - 1$, models are re-fit periodically as time advances, and one fixed 30% tail of the sample is the common test window for every model.

Q. *How can GARCH be refit only monthly and still produce daily forecasts?*

A. Parameters and states are separate. Parameters are re-estimated every 22 days on an expanding window; in between, the fitted recursion is rolled forward daily with observed returns, updating the state σ_t^2 exactly as the model equation prescribes. Daily MLE refits would cost 22 times more and change results negligibly.

Q. *Explain the difference between GARCH, GJR-GARCH and EGARCH in one breath each.*

A. GARCH: tomorrow's variance is a constant plus yesterday's squared return plus yesterday's variance, symmetric in the return's sign. GJR: same, plus an extra term that switches on only when yesterday's return was negative. EGARCH: models log-variance with a signed shock term, so negative shocks raise volatility more and the variance is positive by construction.

Q. *Your winner changed when the sample was extended. Does that undermine the study?*

A. No, it illustrates it. On data to 2024 plain GARCH won and the leverage edge was insignificant; extending to mid-2026 put two asymmetric crashes (2022, April 2025) into the test window, and the leverage models became significantly better ($p < 0.0001$). Which model is best depends on the regimes the evaluation window contains; that dependence is a finding, not an embarrassment, and it is why the report states the window explicitly everywhere.

Q. *Is the LSTM actually good here, or just lucky?*

A. Genuinely competitive, with caveats. It ranks third of eight, ahead of plain GARCH, has the best RMSE, and is statistically indistinguishable from the winner ($p = 0.19$). But indistinguishable is not better: the claim the project supports is that no learned model outranks the leverage GARCH family, while the LSTM's showing weakens the case that deep learning is useless on daily data.

Q. *Why does the LSTM win RMSE but lose QLIKE?*

A. It trains on a smoothed target with a symmetric loss, so it is accurate on average (good RMSE) but reacts a step late at eruptions. QLIKE charges heavily for exactly those late, under-predicted spikes. Different metrics price different errors; the ranking flip between metric families is the trade-off in its purest form.

Q. *Why is the naive baseline's QLIKE four orders of magnitude worse when its RMSE is only modestly worse?*

A. QLIKE contains the ratio proxy/forecast, which diverges when a near-zero forecast meets a non-zero realization. The naive forecast (yesterday's squared return) produces near-zero forecasts after every flat day, so it accumulates enormous penalties. RMSE, being symmetric and additive, hides this failure mode.

Q. *What is the Diebold–Mariano test and why the Harvey correction?*

A. DM tests whether the mean of the daily loss differential between two models is zero, using its autocorrelation-consistent variance. The Harvey–Leybourne–Newbold correction adjusts the statistic's small-sample size and compares against a t distribution, avoiding over-rejection in finite samples. It converts leaderboard gaps into claims with error bars.

Q. *Name the anti-leakage guards in the LSTM pipeline.*

A. Four. Sequences are labelled by their final row, so no input window extends past its label date. Feature standardization uses training-window means and standard deviations only. The early-stopping validation slice comes from the end of the training window, never from test. And the forward-looking target forces a 5-day buffer between any training row and the forecast date.

Q. *Why is the LSTM so small: one layer, 32 units?*

A. Because the training set is roughly 2,800 noisy sequences. Model capacity has to be paid for with data; a deep stack would memorize noise. The honest question is whether any recurrence beats GARCH, not how large a network can be built, and early stopping (epoch 37 of 200) confirms the ceiling is data, not epochs.

Q. *Why did you reject warm-starting XGBoost between refits?*

A. It either grows the ensemble at every refit or changes which trees exist, so predictions drift run-to-run for a saving of a couple of seconds per refit. From-scratch refits keep the study exactly reproducible; correctness and reproducibility beat a micro-optimization.

Q. *How is the pipeline reproducible and fast?*

A. All seeds fixed (42), one shared split, package versions pinned in the report, and every reported number regenerated end-to-end from the committed code. The five expensive fits (three GARCH variants, XGBoost, LSTM) are mutually independent, so they run in parallel worker processes; the full study reproduces in about one minute.

Q. *What was the single most important methodological lesson?*

A. That the harness matters more than the model. Building a genuinely leak-free walk-forward evaluation (target buffering, train-only scaling, end-of-train validation) was harder and more consequential than any modelling choice, and the choice of metric changed the ranking of models. “Better” is undefined until the loss function and the evaluation window are fixed.

Artifacts: code and results at <https://github.com/iitgF/T9>; full report `report.tex`; short report `report_simple.tex`; condensed memo `memo_condensed.tex`.